

코드 설명서 (v3 · 전체 파일판)

프로그래밍을 한 번도 안 해본 분도 27개 자바 파일 전부를 따라 읽을 수 있도록 만들었습니다

작성일 2026-07-03 · 작성자 원석(BE) · 대상: PM · 디자인 · 비개발 팀원 · 코딩 입문자

이 문서는 자바나 스프링을 몰라도 "이 파일이 왜 있고, 이 줄이 무슨 뜻인지"를 따라 읽을 수 있도록 만들었습니다. 실제 코드와 100% 동일하며, 코드 옆에 바로 설명을 붙였습니다. v2까지는 대표 파일 8개만 깊게 다뤘지만, 이번 v3에서는 프로그래밍 기초 개념부터 시작해서 이 프로젝트의 자바 파일 27개 전부를 빠짐없이 설명합니다.

0. 시작하기 전에 — 프로그래밍 아주 기초

자바나 코딩을 한 번도 해본 적이 없다면 이 부분만 먼저 읽어주세요. 이 문서를 따라오는 데 필요한 만큼만, 딱 5가지만 담았습니다. 이미 코딩을 아신다면 1장부터 바로 읽으셔도 됩니다.

① 변수 — 이름표가 붙은 상자

변수는 값을 담아두는 상자입니다. 상자마다 이름(변수명)과 어떤 종류의 물건을 담을지(자료형)가 정해져 있습니다.

```
String email = "a@a.com"; // email이라는 상자에 글자(String) 담음
int age = 20; // age라는 상자에 숫자(int) 담음
```

이 문서에서 자주 나오는 자료형: String(글자) · int·Long(숫자) · boolean(참 또는 거짓, true/false)

② 클래스와 객체 — 봉어빵 틀과 봉어빵

클래스(class)는 봉어빵 틀과 같습니다 — "어떤 모양의 데이터를 만들지"에 대한 설계도입니다. 그 틀로 실제로 찍어낸 봉어빵 하나하나가 객체(object)입니다. User라는 클래스(틀)로 회원 한 명 한 명(객체)을 계속 찍어낼 수 있는 것과 같습니다.

③ 메서드(함수) — 동작을 담은 덩어리

특정 동작을 모아둔 덩어리입니다. 필요한 재료(매개변수)를 받아서 일을 처리하고, 결과물(반환값)을 돌려줍니다.

```
public String greet(String name) {
    return "안녕, " + name;
}
// greet("철수")를 호출하면 "안녕, 철수"를 돌려받음
```

④ 조건문 — 만약 ~라면

if는 "조건에 따라 다른 동작을 하라"는 뜻입니다. 이 프로젝트 코드에는 if (프로필이 이미 있으면) { 에러를 던진다 } 같은 형태로 자주 나옵니다.

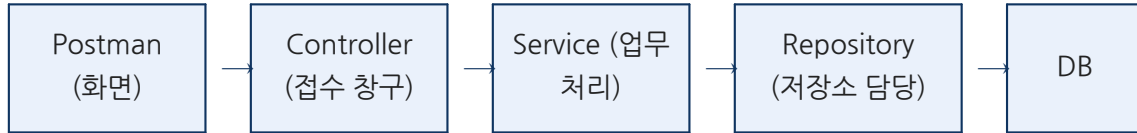
⑤ 반복 — 목록의 각 항목마다

여러 개를 하나씩 처리할 때, 이 프로젝트는 전통적인 for 대신 스트림(stream)이라는 방식을 씁니다. "목록에 있는 것 하나하나에 대해 이 작업을 해줘"라는 뜻이라고 생각하면 됩니다 (6장에서 실제 코드로 다시 설명합니다).

이 5가지만 알면 이 문서를 끝까지 따라올 수 있습니다. 모르는 용어가 또 나오면 2장 용어 사전을 참고하세요.

1. 큰 그림 — 이 프로그램은 어떻게 동작하나

카페에 비유하면 이렇습니다. 손님이 키오스크에서 주문하면(Controller가 요청을 접수), 바리스타가 레시피대로 음료를 만들고(Service가 업무 처리), 재료가 필요하면 창고 담당자에게 요청해서 꺼내오고(Repository가 DB에 접근), 실제 재료와 재고 장부가 있고(Entity = DB에 저장되는 데이터의 설계도), 주문서·영수증 양식이 정해져 있습니다(DTO = 요청/응답 데이터 모양).



응답은 반대 방향(DB → Repository → Service → Controller → Postman)으로 돌아옵니다. 예를 들어 "프로필 조회" 요청 하나가 들어오면, 실제로는 이렇게 5단계를 거칩니다:

단계	담당 파일	하는 일
1	ProfileController.java	GET /profiles/1 요청을 받음
2	ProfileService.java	"1번 프로필을 찾아라"는 업무 지시를 받음
3	ProfileRepository.java	실제 DB에 "id가 1인 프로필 주세요" SQL을 날림
4	(DB)	저장된 데이터를 돌려줌
5	ProfileResponse.java	DB에서 나온 원본 데이터를 화면에 보낼 안전한 모양으로 포장

2. 미리 알아두면 좋은 용어

용어	쉬운 설명
API	프로그램끼리 데이터를 주고받기 위한 약속된 창구. "이 주소로 이렇게 요청하면 이런 응답을 주겠다"는 규칙
엔드포인트	API의 구체적인 주소 하나. 예: GET /profiles/1
HTTP 메서드	요청의 종류. GET(조회) · POST(생성) · PUT(전체 수정) · PATCH(일부 수정) · DELETE(삭제)
JSON	{"이름": "값"} 형태로 데이터를 주고받는 공용 표기법. 거의 모든 API가 이 형식을 씀
어노테이션 (@)	코드 위에 @무언가 형태로 붙이는 "표시". 스프링이 이 표시를 보고 자동으로 동작을 붙여줌
엔티티 (Entity)	DB 테이블 하나에 대응하는 자바 클래스. "이 데이터가 DB에 이런 모양으로 저장된다"는 설계도
리포지토리 (Repository)	DB에 저장하거나 꺼내오는 일만 전담하는 부품
서비스 (Service)	"이 요청이 들어오면 어떤 순서로 무엇을 확인하고 처리할지" 실제 업무 규칙이 담긴 부품
컨트롤러 (Controller)	외부(Postman, 화면)의 요청을 가장 먼저 받는 창구 부품
DTO	Data Transfer Object. 요청/응답으로 주고받는 데이터의 "양식지". 엔티티를 그대로 보내지 않고 이 양식으로 한 번 걸러서 보냄
예외 (Exception)	프로그램 실행 중 생기는 "문제 상황". 예: 존재하지 않는 프로필을 조회하려고 함
패키지 (package)	자바 코드를 담는 폴더 개념. 관련 있는 파일끼리 같은 패키지(폴더)에 모아둠
Lombok	반복되는 코드(getter, 생성자 등)를 어노테이션 한 줄로 자동 생성해주는 도구
enum	미리 정해진 값 중 하나만 고를 수 있게 만든 목록 타입. 예: 성적은 A+, A, B+ ... 중 하나

3. 폴더(패키지) 구조 — 왜 이렇게 나뉘었나

"기능(도메인)" 단위로 폴더를 나눴습니다. 회원(auth), 프로필(profile), 수강 이력(course), 졸업 인증(certification)이 각각 독립된 폴더를 갖고, 그 안에 다시 controller · service · domain · repository · dto가 있습니다. 이렇게 하면 "수강 이력 관련 코드를 고치고 싶다"고 할 때 course 폴더 하나만 보면 됩니다.

```
com.passport
├── PassportApplication.java (프로그램 시작점)
├── global (여러 도메인이 공통으로 쓰는 것)
│   ├── common/ApiResponse.java (모든 응답의 공통 포장지)
│   └── error/ (에러 처리 3종 세트)
├── auth (회원) domain · repository
├── profile (프로필) controller · service · domain · repository · dto
├── course (수강 이력) controller · service · domain · repository · dto
└── certification (졸업 인증) controller · service · domain · repository · dto
```

auth(회원) 폴더에는 아직 controller가 없습니다. 로그인/회원가입 기능(2단계)이 아직 만들어지지 않았기 때문입니다. 지금은 서버를 켤 때 테스트용 회원 2명이 자동으로 미리 등록됩니다.

4. 핵심 패턴 8개 파일 (한 줄 한 줄 설명)

아래 8개 파일은 각 "부품 종류"를 대표하는 예시입니다. 이 8개를 먼저 이해하면 5~7장에서 나오는 나머지 19개 파일도 훨씬 빠르게 읽힙니다. (나머지 파일도 전부 빠짐없이 다룹니다 — 건너뛰지 않습니다.)

4-1. PassportApplication.java — 프로그램의 시작점

PassportApplication.java

com.passport

서버를 켜는 순간 자바가 가장 먼저 실행하는 파일입니다.

코드	설명
<code>package com.passport;</code>	이 파일이 com.passport 폴더(패키지)에 속한다는 선언
<code>import org.springframework.boot.SpringApplication;</code>	서버를 실제로 켜주는 스프링 부트 도구를 가져옴
<code>import org.springframework.boot.autoconfigure.SpringBootApplication;</code>	"여기가 시작점"이라는 표시(@SpringBootApplication)를 쓰기 위해 가져옴
<code>@SpringBootApplication</code>	이 클래스가 스프링 부트 앱의 대문이라는 표시. 필요한 설정을 스프링이 알아서 찾아줌
<code>public class PassportApplication {</code>	PassportApplication이라는 이름의 설계도(클래스) 시작
<code>public static void main(String[] args) {</code>	자바 프로그램은 항상 이 main에서부터 실행을 시작함
<code>SpringApplication.run(PassportApplication.class, args);</code>	실제로 서버를 켜는 한 줄. 이게 실행되면 8080 포트에서 대기 시작
<code>}</code> <code>}</code>	메서드와 클래스를 닫는 괄호

만약 @SpringBootApplication을 지운다면? 스프링이 "여기가 시작점"이라는 걸 모르게 되어 컴포넌트 스캔(@Service, @RestController 등을 자동으로 찾아 등록하는 과정) 자체가 안 됩니다. 즉 지금까지 만든 컨트롤러·서비스가 전부 무시된 채 텅 빈 서버만 켜집니다. 이 한 줄이 프로젝트 전체를 스프링에게 "소개"해주는 셈입니다.

4-2. User.java — 엔티티 (데이터의 설계도)

User.java

com.passport.auth.domain

회원 정보가 DB에 어떤 모양으로 저장되는지 정의합니다.

코드	설명
<code>package com.passport.auth.domain;</code>	auth(회원) 도메인의 domain(설계도) 폴더
<code>import jakarta.persistence.*; import lombok.*; import org.hibernate.annotations.CreationTimestamp; import java.time.LocalDateTime;</code>	DB 매핑 도구, 반복 코드를 줄여주는 Lombok, 날짜/시간 도구를 가져옴
<code>@Entity</code>	"이 클래스는 DB 테이블과 연결된다"는 표시
<code>@Table(name = "users")</code>	실제로 연결될 테이블 이름은 users
<code>@Getter</code>	모든 필드마다 손으로 getEmail() 같은 함수를 안 만들어도 자동 생성(Lombok)
<code>@NoArgsConstructor(access = AccessLevel.PROTECTED)</code>	빈 User 객체를 아무 데서나 못 만들게 막음(DB 처리용으로만 허용)
<code>public class User {</code>	회원 데이터의 설계도 시작
<code> @Id</code>	이 필드가 테이블의 고유 번호(기본키)
<code> @GeneratedValue(strategy = GenerationType.IDENTITY)</code>	번호를 DB가 1, 2, 3 ... 자동으로 매기게 함
<code> private Long id;</code>	회원 고유 번호
<code> @Column(nullable = false, unique = true) private String email;</code>	필수이고 중복 불가능한 이메일 칸
<code> /** BCrypt 해시 — 평문 저장 금지 */ @Column(nullable = false) private String password;</code>	암호화된 비밀번호 저장 칸 (주석은 개발자에게 남기는 주의 메모)
<code> @Column(nullable = false) private String nickname;</code>	회원가입 시 입력하는 닉네임
<code> @CreationTimestamp @Column(nullable = false, updatable = false) private LocalDateTime createdAt;</code>	저장되는 순간 "지금 시각"이 자동으로 채워지고, 이후 수정은 불가능
<code> @Builder public User(String email, String password, String nickname) { this.email = email; this.password = password; this.nickname = nickname; }</code>	User.builder().email(...).password(...).nickname(...).build() 형태로 편하게 객체를 만드는 방법을 제공. id-createdAt은 사람이 안 넣어도 자동으로 채워짐

왜 @NoArgsConstructor를 public이 아니라 protected로 열었을까요? public이었다면 프로젝트 어디서든 new User()로 email-password도 없는 "깨진" 사용자를 만들 수 있게 됩니다. JPA(스프링이 내부적으로 객체를 조립할

때)만 이 빈 생성자를 쓰도록 좁혀서, 사람이 쓰는 정상 경로는 반드시 @Builder를 거치도록 강제한 것입니다.

@Column(unique = true)는 왜 필요할까요? 나중에 회원가입 로직에서도 "이미 있는 이메일인가?"를 코드로 먼저 확인하겠지만, 그 확인과 실제 저장 사이의 아주 짧은 순간에 같은 이메일로 두 요청이 동시에 들어오면 코드 레벨 검사만으로는 중복을 100% 못 막을 수 있습니다. DB가 마지막 방어선으로 한 번 더 막아주는 것입니다.

4-3. UserRepository.java — 저장소 담당

UserRepository.java

com.passport.auth.repository

회원 데이터를 DB에 저장·조회하는 역할만 전담합니다.

코드	설명
<code>package com.passport.auth.repository;</code>	auth 도메인의 repository(저장소) 폴더
<code>import com.passport.auth.domain.User;</code> <code>import</code> <code>org.springframework.data.jpa.repository.JpaRepository;</code> <code>import java.util.Optional;</code>	위에서 만든 User 설계도와, "저장/조회/삭제" 기본 기능을 이미 다 만들어놓은 스프링 도구를 가져옴
<code>public interface UserRepository extends</code> <code>JpaRepository<User, Long> {</code>	JpaRepository를 물려받는 순간 save(), findById(), delete() 같은 기본 기능이 전부 공짜로 생김. <User, Long>은 "User 테이블을 Long 타입 id로 다룬다"는 뜻
<code>Optional<User> findByEmail(String email);</code>	몸통(구현) 코드가 하나도 없는데도, 메서드 이름만 이렇게 지으면 스프링이 "email로 찾아줘"라는 SQL을 알아서 만들어줍니다. 이게 Spring Data JPA의 핵심 마법입니다
<code>boolean existsByEmail(String email);</code>	그 이메일이 이미 존재하는지 참/거짓만 빠르게 확인
<code>}</code>	인터페이스 끝

interface 안에 코드(몸통)가 없는 게 정상입니다. Spring Data JPA가 메서드 이름을 해석해서 실행 시점에 자동으로 구현을 만들어 끼워 넣기 때문입니다.

findByEmail(String email)이 실제로 어떤 SQL로 바뀌는지 보면 이해가 쉽습니다:

```
SELECT * FROM users WHERE email = ?
```

이 한 줄짜리 SQL을 메서드 이름만 규칙에 맞게(findBy + 필드명) 지으면 Spring Data JPA가 자동으로 만들어 실행해줍니다. 만약 SQL을 직접 쓰는 방식이었다면, 저장소 하나당 이런 코드가 수십 줄씩 반복됐을 것입니다 — 이게 왜 "보일러플레이트(반복 코드)를 줄여준다"고 하는지의 이유입니다.

4-4. DTO — 주고받는 데이터의 "양식지"

ProfileCreateRequest.java

com.passport.profile.dto

프로필 생성 요청 시 Postman/화면이 보내야 하는 데이터 양식입니다.

코드	설명
<pre>public record ProfileCreateRequest(Long userId, String deptCode, String studentId, int admissionYear, String name) { }</pre>	record는 "데이터만 담는 상자"를 짧게 만드는 자바 문법. Postman이 보낸 JSON이 이 상자의 필드 이름과 자동으로 매칭되어 객체로 변환됨
Long userId,	어떤 회원의 프로필인지
String deptCode,	학과 코드
String studentId,	학번
int admissionYear,	입학연도
String name	이름(선택)
) { }	상자 정의 끝

ProfileResponse.java

com.passport.profile.dto

프로필 조회/생성 결과로 화면에 돌려줄 데이터 양식입니다.

코드	설명
<pre>public record ProfileResponse(Long id, Long userId, String deptCode, String studentId, int admissionYear, String name) { public static ProfileResponse from(Profile profile) { return new ProfileResponse(profile.getId(), profile.getUser().getId(), profile.getDeptCode(), profile.getStudentId(), profile.getAdmissionYear(), profile.getName()); } }</pre>	화면에 보낼 6개 항목을 담는 상자
public static ProfileResponse from(Profile profile) {	DB에서 막 꺼낸 원본 데이터(Profile 엔티티)를 화면용 상자로 "번역"하는 함수
return new ProfileResponse(profile.getId(), profile.getUser().getId(), profile.getDeptCode(), profile.getStudentId(), profile.getAdmissionYear(), profile.getName()); }	엔티티의 각 값을 꺼내서 새 상자에 옮겨 담음

왜 엔티티를 그대로 보내지 않고 DTO로 한 번 "번역"할까요? User 엔티티에는 비밀번호 같은 민감한 정보도 같이 들어있는데, 실수로 그게 화면까지 새어나가는 것을 막기 위해서입니다. DTO는 "정말 화면에 필요한 것만 딱 골라 담은 안전한 창구" 역할을 합니다.

구체적으로 어떤 사고가 날 수 있었을까요? 만약 ProfileResponse 대신 Profile 엔티티를 그대로 반환했다면, Profile은 User 객체를 통째로 들고 있어서(profile.getUser()) 그 User 안의 password 필드까지 JSON으로 같이 새어나갈 위험이 있습니다. DTO를 한 번 거치면 "id, userId, deptCode, studentId, admissionYear, name" 딱 6개만 내보내도록 처음부터 차단됩니다.

4-5. ProfileService.java — 실제 업무 처리 담당

ProfileService.java

com.passport.profile.service

"프로필을 만들어라/찾아라/고쳐라"는 요청이 오면 실제 규칙대로 처리합니다. (프로필 생성 부분만 발췌)

코드	설명
@Service	"이 클래스는 업무 처리를 담당하는 부품"이라고 스프링에 등록
@RequiredArgsConstructor	아래 final로 선언된 두 저장소를 자동으로 연결해주는 생성자를 Lombok이 만들어줌
@Transactional(readOnly = true)	기본적으로 "조회만 하는 안전 모드"로 동작 (실수로 데이터가 바뀌는 것을 예방)
public class ProfileService {	프로필 업무 처리 설계도 시작
private final ProfileRepository profileRepository; private final UserRepository userRepository;	프로필 저장소, 회원 저장소를 각각 연결
@Transactional	이 메서드는 데이터를 "바꾸는" 작업이라 조회 전용 모드를 해제
public ProfileResponse create(ProfileCreateRequest request) {	"프로필을 새로 만들어라"는 요청을 받는 함수
User user = userRepository.findById(request.userId()) .orElseThrow(() -> new BusinessException(ErrorCode.USER_NOT_FOUND));	요청에 적힌 userId로 실제 회원이 있는지 찾아보고, 없으면 "회원 없음" 에러를 던지고 즉시 중단
if (profileRepository.existsByUserId(user.getId())) { throw new BusinessException(ErrorCode.PROFILE_ALREADY_EXISTS); }	이미 그 회원의 프로필이 있다면 "이미 있음" 에러를 던지고 중단 (회원 1명당 프로필 1개 규칙)
Profile profile = Profile.builder() .user(user) .deptCode(request.deptCode()) .studentId(request.studentId()) .admissionYear(request.admissionYear()) .name(request.name()) .build();	여기까지 문제가 없으면 새 프로필 객체를 조립
return ProfileResponse.from(profileRepository.save(profile)); }	DB에 실제로 저장하고, 저장 결과를 화면용 모양(DTO)으로 바꿔서 반환

get()과 update() 메서드도 구조는 같습니다 — "먼저 존재 여부를 확인하고, 없으면 에러, 있으면 처리"라는 같은 패턴을 반복합니다.

@Transactional이 없다면 무슨 일이 생길까요? create() 안에서 프로필을 저장한 직후 다른 이유로 예외가 발생했다고 가정하면, @Transactional이 있을 때는 그 메서드 안에서 있었던 DB 변경 전부가 자동으로 취소(롤백)됩니다. 데이터가 "반만 저장된" 상태로 남는 사고를 막아주는 것입니다. orElseThrow()도 같은 맥락의 습관입니다 — 문제가 있으면 그 즉시 예외를 던져서 이후 코드가 잘못된 상태로 계속 실행되는 것을 막습니다. 이런 습관을 "빠른 실패(fail fast)"라고 부릅니다.

4-6. ProfileController.java — 외부 요청 접수 담당

ProfileController.java

com.passport.profile.controller

Postman/화면에서 오는 HTTP 요청을 가장 먼저 받는 창구입니다.

코드	설명
<pre>/** * 인증(JWT) 불기 전 임시 테스트용 컨트롤러 — 소유권 검증 없음. * 2단계(인증) 완료 후 로그인 사용자 본인 확인 로직이 추가될 예정. */</pre>	이 클래스가 아직 로그인 검증이 없는 임시 버전이라는 걸 코드에 직접 남긴 메모(자바독 주석)
<pre>@RestController</pre>	"이 클래스는 요청을 받아 JSON으로 응답하는 창구"라는 표시
<pre>@RequestMapping("/api/v1/profiles")</pre>	이 창구의 기본 주소는 /api/v1/profiles
<pre>@RequiredArgsConstructor</pre>	아래 서비스를 자동으로 연결해주는 생성자를 Lombok이 만들어줌
<pre>public class ProfileController {</pre>	프로필 창구 설계도 시작
<pre> private final ProfileService profileService;</pre>	실제 업무 처리는 서비스에게 맡김 (창구 직원은 판단하지 않고 전달만)
<pre> @PostMapping @ResponseStatus(HttpStatus.CREATED)</pre>	POST 방식으로 이 주소에 요청이 오면 아래 함수 실행, 성공 시 201(생성됨) 상태코드
<pre> public ApiResponse<ProfileResponse> create(@RequestBody ProfileCreateRequest request) {</pre>	@RequestBody: Postman이 보낸 JSON을 자동으로 ProfileCreateRequest 객체로 변환해서 받음
<pre> return ApiResponse.success(profileService.create(request)); }</pre>	서비스에게 실제 처리를 맡기고, 결과를 공통 포장지(ApiResponse)에 담아 반환
<pre> @GetMapping("/{id}")</pre>	GET 방식으로 /profiles/123 같은 주소로 오면 실행. {id}는 주소의 일부를 값처럼 받겠다는 뜻
<pre> public ApiResponse<ProfileResponse> get(@PathVariable Long id) {</pre>	@PathVariable: 주소의 {id} 부분을 실제 값으로 꺼내 옴
<pre> return ApiResponse.success(profileService.get(id)); }</pre>	조회 결과 반환

```

@PatchMapping("/{id}")
public ApiResponse<ProfileResponse> update(@PathVariable
Long id, @RequestBody ProfileUpdateRequest request) {
    return ApiResponse.success(profileService.update(id,
request));
}
}

```

수정 요청도 같은 패턴: 주소의 id
+ 요청 바디를 받아 서비스에 위임

GET은 성공하면 보통 200(OK), POST로 새로 만들었을 때는 201(Created)을 씁니다 — 둘 다 "성공"이지만 "이미 있던 걸 보여준 것"과 "새로 만든 것"을 구분해주는 국제 표준 약속입니다. 그리고 이 컨트롤러 코드에는 판단 로직(존재 확인, 중복 확인 등)이 하나도 없이 그냥 서비스를 부르기만 합니다. 이렇게 컨트롤러를 "얇게" 유지하면, 나중에 테스트할 때도 실제 HTTP 요청 없이 ProfileService만 따로 테스트할 수 있어서 훨씬 빠르고 간단해집니다.

4-7. 에러 처리 3종 세트

문제가 생겼을 때(예: 없는 프로필 조회) 서버가 어떻게 반응할지 미리 정해둔 3개의 파일입니다. 이 셋이 함께 움직여서 "어떤 에러든 항상 같은 모양으로 응답한다"를 보장합니다.

ErrorCode.java

com.passport.global.error

우리 서비스에서 날 수 있는 모든 에러의 "목록표"입니다.

코드	설명
<pre>public enum ErrorCode {</pre>	enum(열거형): 미리 정해진 값 중 하나만 고를 수 있는 목록. 여기서는 "에러 종류 전체 목록"
<pre> // Common INVALID_INPUT_VALUE(HttpStatus.BAD_REQUEST, "COMMON-001", "입력값이 올바르지 않습니다."), ENTITY_NOT_FOUND(HttpStatus.NOT_FOUND, "COMMON-002", "요청한 리소스를 찾을 수 없습니다."), ACCESS_DENIED(HttpStatus.FORBIDDEN, "COMMON-003", "접근 권한이 없습니다."), INTERNAL_SERVER_ERROR(HttpStatus.INTERNAL_SERVER_ERROR, "COMMON-004", "서버 내부 오류가 발생했습니다."),</pre>	아직 어느 도메인에도 속하지 않는 공통 에러 4가지. 4번(서버 내부 오류)이 바로 GlobalExceptionHandler의 마지막 그룹이 쓰는 값입니다
<pre> // Auth (다음 단계에서 사용) EMAIL_DUPLICATED(HttpStatus.CONFLICT, "AUTH-001", "이미 사용 중인 이메일입니다."), INVALID_CREDENTIALS(HttpStatus.UNAUTHORIZED, "AUTH-002", "이메일 또는 비밀번호가 올바르지 않습니다."), UNAUTHENTICATED(HttpStatus.UNAUTHORIZED, "AUTH-003", "인증이 필요합니다."), USER_NOT_FOUND(HttpStatus.NOT_FOUND, "AUTH-004", "회원을 찾을 수 없습니다."),</pre>	회원 관련 에러 4가지. USER_NOT_FOUND는 지금 ProfileService.create()에서 실제로 쓰이고, 나머지 3개는 2단계(로그인) 만들 때 쓸 예정으로 미리 준비해둔 것
<pre> // Profile PROFILE_NOT_FOUND(HttpStatus.NOT_FOUND, "PROFILE-001", "프로필을 찾을 수 없습니다."), PROFILE_ALREADY_EXISTS(HttpStatus.CONFLICT, "PROFILE-002", "이미 프로필이 등록되어 있습니다."), PROFILE_ACCESS_DENIED(HttpStatus.FORBIDDEN, "PROFILE-003", "본인의 프로필만 접근할 수 있습니다."),</pre>	프로필 관련 에러 3가지. 앞의 2개는 지금 ProfileService에서 실제로 쓰이고, PROFILE_ACCESS_DENIED(본인 소유 아님)는 2단계에서 소유권 검증을 붙일 때 쓸 예정
<pre> // Course COURSE_NOT_FOUND(HttpStatus.NOT_FOUND, "COURSE-001", "수강 이력을 찾을 수 없습니다."),</pre>	CourseService.findCourse()에서 실제로 쓰이는 에러
<pre> // Certification CERTIFICATION_NOT_FOUND(HttpStatus.NOT_FOUND, "CERT-001", "졸업 인증 정보를 찾을 수 없습니다.");</pre>	마지막 항목은 세미콜론(;)으로 목록을 닫습니다 — 지금 코드에선 실제로 던져지는 곳은 없지만, 나중에 개별 인증 조회 API가 생기면 쓰일 걸 대비해 미리 만들어둔 값

<pre>private final HttpStatus status; private final String code; private final String message;</pre>	위에서 정의한 (상태코드, 코드, 메시지) 3개를 각 enum 값이 저장할 상자
<pre>ErrorCode(HttpStatus status, String code, String message) { this.status = status; this.code = code; this.message = message; }</pre>	목록의 각 항목에 값 3개를 채워 넣는 생성자
<pre>public HttpStatus getStatus() { return status; } public String getCode() { return code; } public String getMessage() { return message; }</pre>	나중에 이 예외의 상태코드·코드·메시지를 각각 꺼내볼 수 있는 함수 3개. GlobalExceptionHandler가 이 함수들을 호출해서 응답을 만들

BusinessException.java

com.passport.global.error

"우리 서비스에서 의도적으로 발생시키는 문제 상황"을 표현하는 전용 예외입니다.

코드	설명
<pre>public class BusinessException extends RuntimeException {</pre>	자바가 원래 갖고 있는 "문제 상황(예외)" 기능을 물려받아 우리 서비스 전용으로 만들
<pre>private final ErrorCode errorCode;</pre>	이 예외가 목록표(ErrorCode)의 어떤 항목에 해당하는지 저장
<pre>public BusinessException(ErrorCode errorCode) { super(errorCode.getMessage()); this.errorCode = errorCode; }</pre>	가장 많이 쓰는 형태 — ErrorCode 하나만 넘기면, 그 예외의 기본 메시지를 그대로 씬. super(...)는 부모(RuntimeException)에게 "이 예외의 메시지는 이거야"라고 전달하는 것
<pre>public BusinessException(ErrorCode errorCode, String message) { super(message); this.errorCode = errorCode; }</pre>	메시지를 직접 지정하고 싶을 때 쓰는 두 번째 형태(오버로딩: 같은 이름의 생성자를 매개변수만 다르게 여러 개 만드는 것)

GlobalExceptionHandler.java

com.passport.global.error

프로젝트 어디서 예외가 나든 이 한 곳에서 모아 처리합니다.

코드	설명
<pre>@Slf4j @RestControllerAdvice public class GlobalExceptionHandler {</pre>	@Slf4j: 로그(기록)를 남길 수 있는 도구(log 변수)를 자동으로 준비해줌(Lombok). @RestControllerAdvice: "프로젝트 전체에서 발생하는 예외를 여기서 한 곳에 모아 처리하겠다"는 표시
<pre> public record ErrorResponse(String code, String message) { public static ErrorResponse of(ErrorCode errorCode) { return new ErrorResponse(errorCode.getCode(), errorCode.getMessage()); } public static ErrorResponse of(ErrorCode errorCode, String message) { return new ErrorResponse(errorCode.getCode(), message); } }</pre>	에러가 났을 때 화면에 보낼 응답 모양(코드+메시지). of()는 ErrorCode로부터 이 응답을 편하게 만드는 지름길 함수 — 메시지를 안 주면 ErrorCode의 기본 메시지를, 주면 그 메시지를 씀
<pre> @ExceptionHandler(BusinessException.class) public ResponseEntity<ErrorResponse> handleBusinessException(BusinessException e) { ErrorCode errorCode = e.getErrorCode(); return ResponseEntity.status(errorCode.getStatus()) .body(ErrorResponse.of(errorCode, e.getMessage())); }</pre>	"BusinessException이 터지면 이 함수가 대신 처리한다"는 표시. 에러 종류에 맞는 HTTP 상태코드(404, 409 등)로 응답
<pre> @ExceptionHandler(MethodArgumentNotValidException.class) public ResponseEntity<ErrorResponse> handleMethodArgument NotValidException(MethodArgumentNotValidException e) { String message = e.getBindingResult().getFieldErrors().stream() .findFirst() .map(FieldError::getDefaultMessage) .orElse(ErrorCode.INVALID_INPUT_VALUE.getMessage()); ; return ResponseEntity.status(HttpStatus.BAD_REQUEST) .body(ErrorResponse.of(ErrorCode.INVALID_INPUT_VAL UE, message)); }</pre>	요청 바디에 @Valid 같은 유효성 검사를 걸어놨는데 그 검사에 실패했을 때 스프링이 자동으로 던지는 예외를 처리. 여러 항목이 동시에 잘못됐을 수 있어서, 그중 첫 번째 에러 메시지(findFirst)만 골라 응답 — 지금 이 프로젝트는 아직 @Valid를 실제로 걸어둔 곳은 없지만, 나중에 쓸 걸 대비해 미리 준비해둔 핸들러
<pre> @ExceptionHandler(ConstraintViolationException.class) public ResponseEntity<ErrorResponse> handleConstraintViolationException(ConstraintViolationException e) { return ResponseEntity.status(HttpStatus.BAD_REQUEST) .body(ErrorResponse.of(ErrorCode.INVALID_INPUT_VAL UE, e.getMessage())); }</pre>	위와 비슷하지만 다른 종류의 유효성 검사 실패(예: 메서드 파라미터에 직접 건 검증)를 처리하는 핸들러. 이것도 아직 실제로 걸리는 경우는 없지만 대비용
<pre> @ExceptionHandler(Exception.class) public ResponseEntity<ErrorResponse> handleException(Exception e) { log.error("예상하지 못한 예외 발생", e); return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR) .body(ErrorResponse.of(ErrorCode.INTERNAL_SERVER_E RROR)); }</pre>	위에서 미처 예상 못 한 모든 예외를 마지막으로 받는 그물. 실제 원인은 서버 로그에 자세히 남기고(log.error), 화면에는 안전하게 "서버 내부 오류"라는 뭉뚱그린 메시지만 전달(내부 정보 노출 방지)

만약 `GlobalExceptionHandler`가 없었다면, `ProfileController`·`CourseController`·`CertificationController` 세 곳 모두에서 각자 try-catch로 "프로필이 없으면 404를 만들어서 반환"하는 코드를 반복해서 써야 했을 겁니다. 컨트롤러가 3개뿐인 지금도 번거롭지만, 나중에 인증·진단·추천까지 컨트롤러가 10개 넘게 늘어나면 이 반복은 감당하기 어려워집니다. `@RestControllerAdvice` 하나가 프로젝트를 전체를 "가로채서" 처리해주는 덕분에 각 컨트롤러는 정상적인 경우만 신경 쓰면 됩니다.

실전 사례: 이 프로젝트를 Postman으로 테스트하던 중 실제로 이 `handleException` 함수가 동작해서 500 에러를 반환한 적이 있습니다. 처음엔 `log.error` 줄이 없어서 원인을 못 찾다가, 이 줄을 추가하고 나서야 "curl 명령이 한글을 깨진 형태로 보냈다"는 진짜 원인을 찾을 수 있었습니다. 이렇게 에러를 한 곳에서 기록해두는 것이 실전에서 왜 중요한지 보여주는 사례입니다.

4-8. ApiResponse.java — 모든 응답의 공통 포장지

ApiResponse.java

com.passport.global.common

성공했을 때 모든 API가 똑같은 모양으로 응답하도록 통일합니다.

코드	설명
<pre>public record ApiResponse<T>(boolean success, T data, String message) {</pre>	<T>는 "안에 어떤 타입의 데이터가 들어올지는 나중에 정한다"는 뜻(제네릭). 모든 응답을 (성공여부 / 데이터 / 메시지) 3칸으로 통일
<pre> public static <T> ApiResponse<T> success(T data) { return new ApiResponse<>(true, data, null); }</pre>	성공 응답을 만드는 지름길 함수. success=true, message는 비워둠
<pre> public static ApiResponse<Void> error(String message) { return new ApiResponse<>(false, null, message); }</pre>	실패 응답을 만드는 지름길 함수 (현재는 에러 처리를 GlobalExceptionHandler가 별도로 담당해서 아직 실제로는 안 쓰임)

실제 응답 예시: {"success": true, "data": {...실제 데이터...}, "message": null} — Postman 매뉴얼 문서에서 본 응답들이 전부 이 모양을 따릅니다.

만약 <T> 없이 그냥 Object data로 만들었다면 어떻게 될까요? 컴파일 시점에는 문제가 없어 보이지만, 실제로 data를 꺼내 쓰는 코드에서는 매번 강제로 타입을 변환(캐스팅)해야 하고, 실수로 엉뚱한 타입으로 변환하면 프로그램이 실행 중에 갑자기 죽습니다. ApiResponse<ProfileResponse>처럼 제네릭을 쓰면 "이 안에는 반드시 ProfileResponse만 들어있다"는 걸 컴파일러가 미리 확인해줘서, 그런 실수 자체가 코드를 작성하는 순간에 걸러집니다.

5. Profile 폴더의 나머지 파일

4장에서 ProfileCreateRequest·ProfileResponse·ProfileService(create만)·ProfileController를 봤습니다. 여기서 나머지를 마저 봅니다.

5-1. Profile.java — 프로필 엔티티

Profile.java

com.passport.profile.domain

프로필 정보가 DB에 어떤 모양으로 저장되는지 정의합니다. User.java와 빼대는 같고, 새로운 부분만 짚습니다.

코드	설명
<pre>@Entity @Table(name = "profiles") @Getter @NoArgsConstructor(access = AccessLevel.PROTECTED) public class Profile {</pre>	User.java에서 이미 설명한 것과 똑같은 4가지 표시입니다 (DB 테이블 연결·테이블명·getter 자동생성·빈 생성자 보호)
<pre>@OneToOne(fetch = FetchType.LAZY) @JoinColumn(name = "user_id", nullable = false, unique = true) private User user;</pre>	이 프로필이 어떤 회원(User) 것인지 연결. @OneToOne은 "회원 한 명당 프로필 한 개"라는 뜻이고, unique = true가 그 규칙을 DB 레벨에서 강제합니다
<pre>/** 요건 매칭 키 (예: 빅데이터 인공지능 전공 코드) */ @Column(nullable = false) private String deptCode;</pre>	학과 코드. 주석은 이 값이 나중에 졸업 요건표(requirement)와 매칭하는 키로 쓰인다는 설계 의도를 남긴 메모
<pre>@Column(nullable = false) private String studentId; @Column(nullable = false) private int admissionYear; private String name;</pre>	학번·입학연도는 필수, 이름은 선택(어노테이션 없음 = 비어있어도 됨)
<pre>@Builder public Profile(User user, String deptCode, String studentId, int admissionYear, String name) { this.user = user; this.deptCode = deptCode; this.studentId = studentId; this.admissionYear = admissionYear; this.name = name; }</pre>	User.java와 같은 방식의 생성자 — Profile.builder()....build()로 객체 생성

```

public void update(String deptCode, String studentId, int
admissionYear, String name) {
    this.deptCode = deptCode;
    this.studentId = studentId;
    this.admissionYear = admissionYear;
    this.name = name;
}

```

User.java에는 없던 새 부분:
이미 만들어진 프로필의 값을
바꿔주는 메서드.
ProfileService.update()가 이
메서드를 호출합니다 (5-4에서
다시 나옵니다)

fetch = FetchType.LAZY는 무슨 뜻일까요? "profile.getUser()를 실제로 호출하기 전까지는 회원 정보를 DB에서 미리 가져오지 않는다"는 뜻입니다. 프로필만 필요한 상황에서 매번 회원 정보까지 같이 조회하면 낭비니까, 진짜 필요할 때만 게으르게(Lazy) 가져오도록 한 것입니다.

5-2. ProfileRepository.java

ProfileRepository.java

com.passport.profile.repository

프로필 저장소. UserRepository.java와 똑같은 원리입니다.

코드	설명
public interface ProfileRepository extends JpaRepository<Profile, Long> {	UserRepository와 같은 구조
Optional<Profile> findByUserId(Long userId);	"이 userId를 가진 회원의 프로필을 찾아줘"
boolean existsByUserId(Long userId);	이미 프로필이 있는지 참/거짓 확인 — ProfileService.create()에서 실제로 쓰인 그 메서드입니다

5-3. ProfileUpdateRequest.java

ProfileUpdateRequest.java

com.passport.profile.dto

프로필 수정 요청 시 보내야 하는 데이터 양식입니다.

코드	설명
public record ProfileUpdateRequest(String deptCode, String studentId, int admissionYear, String name) { }	ProfileCreateRequest와 거의 같지만 userId가 없습니다 — URL의 {id}로 "어떤 프로필을 고칠지"가 이미 정해지기 때문에, 수정 요청에는 회원 연결 정보를 다시 보낼 필요가 없습니다

5-4. ProfileService.java — get()·update() 마저 보기

4-5에서는 create()만 봤습니다. 나머지 두 메서드를 마저 보면:

코드	설명
----	----

<pre>public ProfileResponse get(Long profileId) { return ProfileResponse.from(findProfile(profileId)); }</pre>	profileId로 프로필을 찾아서(findProfile) 화면용 모양(DTO)으로 바꿔 반환
<pre>@Transactional public ProfileResponse update(Long profileId, ProfileUpdateRequest request) { Profile profile = findProfile(profileId); profile.update(request.deptCode(), request.studentId(), request.admissionYear(), request.name()); return ProfileResponse.from(profile); }</pre>	프로필을 찾은 뒤, 5-1에서 본 Profile.update() 메서드를 호출해서 값을 바꿈
<pre>private Profile findProfile(Long profileId) { return profileRepository.findById(profileId) .orElseThrow(() -> new BusinessException(ErrorCode.PROFILE_NOT_FOUND)); }</pre>	get()과 update()가 공통으로 쓰는 "찾아보고 없으면 에러" 로직을 하나로 뽑아둔 메서드. private이라 이 클래스 안에서만 쓸 수 있음

update()에서 흥미로운 점: profileRepository.save(profile)을 따로 호출하지 않았는데도 DB가 실제로 업데이트됩니다. @Transactional 메서드 안에서 DB로부터 가져온 엔티티(profile)를 수정하면, 메서드가 끝날 때 JPA가 "어? 값이 바뀌었네?"를 감지해서 자동으로 UPDATE SQL을 날려줍니다. 이것을 "더티 체크(dirty checking)"이라고 부릅니다.

6. Course(수강 이력) 폴더 전체

여기서부터는 Profile에서 배운 패턴(엔티티·리포지토리·DTO·서비스·컨트롤러)이 그대로 반복됩니다. 그래서 이미 배운 부분은 짧게, 새로운 부분만 자세히 봅니다.

6-1. Course.java — 엔티티 + 이수구분·성적 목록

Course.java

com.passport.course.domain

수강 이력이 DB에 어떤 모양으로 저장되는지, 그리고 이수구분·성적 값의 종류까지 정의합니다.

코드	설명
<pre>@Entity @Table(name = "courses") @Getter @NoArgsConstructor(access = AccessLevel.PROTECTED) public class Course {</pre>	User·Profile과 같은 4가지 표시
<pre> @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "profile_id", nullable = false) private Profile profile;</pre>	이 수강 이력이 어떤 프로필 것인지 연결. @ManyToOne은 "여러 개의 수강 이력이 프로필 하나에 몰릴 수 있다"는 뜻입니다 (Profile의 @OneToMany과 다름 — 한 프로필이 과목을 여러 개 들을 수 있으니까)
<pre> @Column(nullable = false) private String name; @Column(nullable = false) private int credit;</pre>	과목명, 학점
<pre> @Enumerated(EnumType.STRING) @Column(nullable = false) private CourseCategory category; @Enumerated(EnumType.STRING) @Column(nullable = false) private Grade grade;</pre>	이수구분·성적. @Enumerated(EnumType.STRING)은 "DB에 저장할 때 0,1,2 같은 숫자가 아니라 MAJOR_REQUIRED 같은 이름 그대로 저장해줘"라는 뜻 — 나중에 DB를 직접 봐도 무슨 값인지 바로 알 수 있습니다
<pre> /** DB 컬럼명은 year_taken — "year"는 H2 예약어라 테이블 생성이 실패해서 컬럼명만 바꿈. JSON 필드명은 그대로 year. */ @Column(name = "year_taken", nullable = false) private int year; @Column(nullable = false) private int semester;</pre>	이수 연도·학기. 자바 필드 이름은 year인데 DB 컬럼명만 year_taken으로 다르게 지정했습니다 — 이유는 주석에 이미 적혀있고, 아래 실전 사례에서 더 자세히 다룹니다

<pre>@Builder public Course(Profile profile, String name, int credit, CourseCategory category, Grade grade, int year, int semester) { this.profile = profile; this.name = name; this.credit = credit; this.category = category; this.grade = grade; this.year = year; this.semester = semester; }</pre>	User·Profile과 같은 패턴의 생성자 — 7개 값을 전부 받아서 필드에 그대로 담음
<pre>public void update(String name, int credit, CourseCategory category, Grade grade, int year, int semester) { this.name = name; this.credit = credit; this.category = category; this.grade = grade; this.year = year; this.semester = semester; }</pre>	Profile.update()와 같은 패턴의 수정 메서드 — profile 필드는 안 바꾸므로 매개변수에 없음(과목을 다른 프로필로 옮기는 기능은 없다는 뜻)

실전 사례: @Column(name = "year_taken")에 왜 이름을 바꿨을까요? 원래는 year 컬럼명을 그대로 쓰려고 했는데, 실제로 서버를 띄워보니 개발 DB(H2)에서 "year"가 예약어(YEAR라는 날짜 함수와 이름이 겹침)라서 테이블 생성 자체가 실패했습니다. 컬럼명만 year_taken으로 바뀌어서 해결했고, 자바 필드명과 API로 주고받는 JSON은 그대로 year를 씁니다 — 코드를 실제로 돌려보지 않았다면 몰랐을 문제입니다.

이 파일 안에는 CourseCategory·Grade라는 두 개의 목록(enum)도 함께 들어있습니다:

코드	설명
<pre>// 이수 구분 public enum CourseCategory { MAJOR_REQUIRED, MAJOR_ELECTIVE, GE_REQUIRED, GE_ELECTIVE, GENERAL_ELECTIVE }</pre>	이수구분 5가지: 전공필수·전공선택·교양필수·교양선택·자유선택 — 이 중 하나만 고를 수 있음
<pre>// 성적 (GPA 환산값 포함). P/NP는 GPA 계산에서 분자·분모 모두 제외 public enum Grade { A_PLUS("A+", 4.5), A("A", 4.0), B_PLUS("B+", 3.5), B("B", 3.0), C_PLUS("C+", 2.5), C("C", 2.0), D_PLUS("D+", 1.5), D("D", 1.0), F("F", 0.0), P("P", null), NP("NP", null); }</pre>	성적 11가지 전체 목록입니다. 각 괄호 안 두 번째 값(4.5, 4.0 ...)이 GPA 계산에 쓸 점수. P(Pass)·NP(Non-Pass)만 null — GPA 계산에서 제외한다는 뜻
<pre>private final String label; private final Double gpaValue; Grade(String label, Double gpaValue) { this.label = label; this.gpaValue = gpaValue; }</pre>	값마다 "화면에 보여줄 표기"(label)와 "GPA 계산에 쓸 점수"(gpaValue) 두 가지를 같이 저장. 이 생성자가 위 11개 값 각각에 대해 한 번씩 자동으로 호출됨

<pre>@JsonValue public String getLabel() { return label; }</pre>	API 응답을 만들 때 A_PLUS라는 내부 이름 대신 이 함수가 돌려주는 "A+"를 대신 씀
<pre>public Double getGpaValue() { return gpaValue; } public boolean isGpaEligible() { return gpaValue != null; }</pre>	GPA 점수를 꺼내는 함수와, 이 성적이 GPA 계산 대상인지 확인하는 함수(P/NP면 false) — 다음 단계인 GPA 트렌드 계산에서 쓰일 예정
<pre>@JsonCreator public static Grade fromLabel(String label) { for (Grade grade : values()) { if (grade.label.equals(label)) { return grade; } } throw new IllegalArgumentException("알 수 없는 성적 표기입니다: " + label); }</pre>	반대로 Postman이 "A+"라는 문자열을 보내면, 11개 값을 하나씩 돌면서(반복문 for) 표기가 일치하는 걸 찾아 A_PLUS enum 값으로 바꿔줍니다. 일치하는 게 없으면 에러. @JsonValue·@JsonCreator 두 어노테이션이 짝을 이뤄서 "내부 이름 ↔ 화면 표기"를 자듯 번역합니다

왜 CourseCategory·Grade를 별도 파일로 안 만들고 Course 클래스 안에 넣었을까요(중첩 enum)? 이 두 값은 Course를 벗어나서 따로 쓰일 일이 거의 없어서, 파일 수를 줄이려고 일부러 Course.CourseCategory / Course.Grade 형태로 안쪽에 넣었습니다.

6-2. CourseRepository.java

CourseRepository.java

com.passport.course.repository

수강 이력 저장소.

코드	설명
<pre>public interface CourseRepository extends JpaRepository<Course, Long> {</pre>	UserRepository·ProfileRepository와 같은 구조
<pre>List<Course> findAllByProfileId(Long profileId);</pre>	이 프로필의 수강 이력을 전부(목록으로) 찾아줌
<pre>Optional<Course> findByIdAndProfileId(Long id, Long profileId);</pre>	findByEmail처럼 필드 하나가 아니라 And로 조건 두 개를 이어 붙인 예. "이 id를 가지면서 동시에 이 profileId에 속한 과목"만 찾음 — 남의 프로필에 있는 과목 id를 넣어도 못 찾게 막는 안전장치입니다

6-3. CourseCreateRequest · CourseUpdateRequest · CourseResponse

CourseCreateRequest.java / CourseUpdateRequest.java / CourseResponse.java

수강 이력 등록·수정·조회용 데이터 양식. ProfileCreateRequest 등과 같은 record 패턴입니다.

코드	설명
<pre>public record CourseCreateRequest(String name, int credit, Course.CourseCategory category, Course.Grade grade, int year, int semester){ }</pre>	새로운 점: 필드 타입으로 Course.CourseCategory·Course.Grade(6-1에서 본 중첩 enum)를 그대로 씁니다. Postman이 category에 "MAJOR_REQUIRED", grade에 "A+"를 문자열로 보내면 자동으로 이 enum 값으로 변환됩니다
<pre>public record CourseUpdateRequest(String name, int credit, Course.CourseCategory category, Course.Grade grade, int year, int semester){ }</pre>	CourseCreateRequest와 필드가 완전히 같습니다 — 수정 시에도 6개 값을 전부 다시 받아 통째로 교체하는 방식(PUT 방식의 특징)
<pre>public record CourseResponse(Long id, String name, int credit, Course.CourseCategory category, Course.Grade grade, int year, int semester){ }</pre>	ProfileResponse와 같은 "엔티티 → DTO 번역" 패턴. id까지 포함해 7개 항목을 화면에 돌려줌
<pre>public static CourseResponse from(Course course) { return new CourseResponse(course.getId(), course.getName(), course.getCredit(), course.getCategory(), course.getGrade(), course.getYear(), course.getSemester()); }</pre>	엔티티의 7개 값을 그대로 꺼내서 새 상자(DTO)에 옮겨담음 — ProfileResponse.from()과 완전히 같은 패턴

6-4. CourseService.java

CourseService.java

com.passport.course.service

등록·목록조회·수정·삭제 4개 메서드 전부를 봅니다.

코드	설명
----	----

<pre> @Service @RequiredArgsConstructor @Transactional(readOnly = true) public class CourseService { </pre>	ProfileService와 같은 3가지 표시
<pre> private final CourseRepository courseRepository; private final ProfileRepository profileRepository; </pre>	수강 이력 저장소, 프로필 저장소를 각각 연결
<pre> @Transactional public CourseResponse create(Long profileId, CourseCreateRequest request) { Profile profile = findProfile(profileId); Course course = Course.builder() .profile(profile) .name(request.name()) .credit(request.credit()) .category(request.category()) .grade(request.grade()) .year(request.year()) .semester(request.semester()) .build(); return CourseResponse.from(courseRepository.save(course)); } </pre>	ProfileService.create()와 같은 패턴: 프로필을 확인하고, 새 Course 객체를 조립해서, 저장 후 DTO로 변환해 반환. 다른 점은 회원 대신 프로필에 소속시킨다는 것
<pre> public List<CourseResponse> list(Long profileId) { findProfile(profileId); return courseRepository.findAllByProfileId(profileId).stream() .map(CourseResponse::from) .toList(); } </pre>	이 프로필이 진짜 있는지만 확인하고(findProfile), DB에서 가져온 Course 목록을 스트림으로 하나씩 CourseResponse로 변환(.map)해서 새 리스트로 만들(.toList()). "목록의 각 항목마다 이 작업을 해줘"가 바로 0장에서 예고한 스트림입니다
<pre> @Transactional public CourseResponse update(Long profileId, Long courseId, CourseUpdateRequest request) { Course course = findCourse(profileId, courseId); course.update(request.name(), request.credit(), request.category(), request.grade(), request.year(), request.semester()); return CourseResponse.from(course); } </pre>	ProfileService.update()와 같은 패턴 — 더티 체킹(5-4에서 배운 개념)으로 자동 저장됨
<pre> @Transactional public void delete(Long profileId, Long courseId) { courseRepository.delete(findCourse(profileId, courseId)); } </pre>	삭제할 과목을 먼저 찾고(없으면 findCourse 안에서 예러), 있으면 그대로 삭제. 반환값이 void(없음)인 이유는 삭제는 "성공했다"는 사실 자체가 결과이기 때문
<pre> private Profile findProfile(Long profileId) { return profileRepository.findById(profileId) .orElseThrow(() -> new BusinessException(ErrorCode.PROFILE_NOT_FOUND)); } </pre>	ProfileService의 같은 이름 메서드와 똑같은 역할(이 클래스 안에서 독립적으로 한 번 더 정의됨)

<pre>private Course findCourse(Long profileId, Long courseId) { return courseRepository.findByProfileIdAndCourseId(courseId, profileId) .orElseThrow(() -> new BusinessException(ErrorCode.COURSE_NOT_FOUND)); }</pre>	6-2에서 본 findByProfileIdAndCourseId를 실제로 사용하는 곳 — 남의 프로필 소속 과목 id를 넣으면 여기서 COURSE_NOT_FOUND 예러가 났
---	--

CourseResponse::from처럼 콜론 두 개(::)로 쓰는 문법은 "메서드 레퍼런스"라고 부릅니다. .map(course -> CourseResponse.from(course))라고 풀어 쓴 것과 완전히 같은 뜻인데, 더 짧게 쓰는 방법입니다.

6-5. CourseController.java

CourseController.java

com.passport.course.controller

4개 엔드포인트 전부를 봅니다. ProfileController와 같은 얇은 창구 패턴입니다.

코드	설명
<pre>/** * 인증(JWT) 불기 전 임시 테스트용 컨트롤러 — 소유권 검증 없음. */ @RestController @RequestMapping("/api/v1/profiles/{profileId}/courses") @RequiredArgsConstructor public class CourseController { private final CourseService courseService; @PostMapping @ResponseStatus(HttpStatus.CREATED) public ApiResponse<CourseResponse> create(@PathVariable Long profileId, @RequestBody CourseCreateRequest request) { return ApiResponse.success(courseService.create(profileId, request)); } @GetMapping public ApiResponse<List<CourseResponse>> list(@PathVariable Long profileId) { return ApiResponse.success(courseService.list(profileId)); } @PutMapping("/{courseId}") public ApiResponse<CourseResponse> update(@PathVariable Long profileId, @PathVariable Long courseId, @RequestBody CourseUpdateRequest request) { return ApiResponse.success(courseService.update(profileId, courseId, request)); } }</pre>	ProfileController와 같은 이유의 메모 + 같은 4가지 표시. 다른 점: 주소 자체에 {profileId}가 포함되어 있어서, 이 컨트롤러의 모든 메서드가 "어떤 프로필의 수강 이력인지"를 항상 받습니다 실제 업무 처리는 서비스에게 위임 ProfileController.create()와 같은 패턴. 다른 점: @PathVariable로 받는 profileId와 @RequestBody를 둘 다 받아서 서비스에 넘김 ProfileController.get()과 다른 점: 주소에 {id}가 따로 없고(@GetMapping에 경로 없음), 하나가 아니라 목록(List)을 반환함 — "이 프로필의 과목 전체를 보여줘"라는 뜻 ProfileController는 PATCH(일부 수정)를 쓰지만 여기는 PUT(전체 교체)을 씁니다 — CourseUpdateRequest가 6개 값을 전부 다시 받는 것과 짜이 맞습니다

```
@DeleteMapping("/{courseId}")
public ApiResponse<Void> delete(@PathVariable Long profileId,
@PathVariable Long courseId) {
    courseService.delete(profileId, courseId);
    return ApiResponse.success(null);
}
```

DELETE 방식 요청 처리.
ApiResponse<Void>는
"데이터는 없지만 성공/실패
여부는 알려준다"는 뜻 —
Void는 자바에서 "진짜 아무
값도 없다"를 표현하는 특수
타입입니다

7. Certification(졸업 인증) 폴더 전체

이 폴더는 "있으면 수정, 없으면 새로 생성"이라는 새로운 패턴(upsert)이 등장합니다. 나머지는 앞서 배운 패턴과 같습니다.

7-1. Certification.java — 엔티티 + 유니크 제약

Certification.java

com.passport.certification.domain

졸업 인증(어학·봉사·논문) 정보가 DB에 어떤 모양으로 저장되는지 정의합니다.

코드	설명
<pre>@Entity @Table(name = "certifications", uniqueConstraints = @UniqueConstraint(columnNames = {"profile_id", "type"})) @Getter @NoArgsConstructor(access = AccessLevel.PROTECTED) public class Certification {</pre>	@Entity·@Getter·@NoArgsConstructor는 앞서 배운 것과 동일. 새로운 부분은 uniqueConstraints — "profile_id + type 조합"에 유니크 제약을 걸었습니다. 한 프로필이 같은 인증 유형(예: LANGUAGE)을 두 번 가질 수 없게 DB가 막아줍니다 (User.email의 unique=true를 필드 2개 조합으로 확장한 버전)
<pre>@ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "profile_id", nullable = false) private Profile profile;</pre>	Course.java와 똑같은 @ManyToOne 패턴 — 한 프로필에 인증 기록이 여러 건(어학·봉사·논문) 달릴 수 있음
<pre>@Enumerated(EnumType.STRING) @Column(nullable = false) private CertificationType type; @Enumerated(EnumType.STRING) @Column(nullable = false) private CertificationStatus status;</pre>	인증 유형·상태. Course.category·Course.grade와 같은 @Enumerated(EnumType.STRING) 패턴 — 아래 두 enum 값 중 하나씩 저장
<pre>public enum CertificationType { LANGUAGE, VOLUNTEER, THESIS }</pre>	인증 유형 3가지 — 필드 없는 가장 단순한 형태의 enum (CourseCategory와 같은 형태)
<pre>public enum CertificationStatus { PASS, FAIL, NOT_SUBMITTED }</pre>	인증 상태 3가지: 통과·불합격·미제출
<pre>@Builder public Certification(Profile profile, CertificationType type, CertificationStatus status) { this.profile = profile; this.type = type; this.status = status; }</pre>	User·Profile·Course와 같은 패턴의 생성자

```
public void updateStatus(CertificationStatus status) {
    this.status = status;
}
```

Profile.update()·Course.update()와 같은 성격의 수정 메서드. 다만 이건 status 하나만 바꿉니다

7-2. CertificationRepository.java

CertificationRepository.java

com.passport.certification.repository

졸업 인증 저장소.

코드	설명
public interface CertificationRepository extends JpaRepository<Certification, Long> {	다른 리포지토리들과 같은 구조
List<Certification> findAllByProfileId(Long profileId);	이 프로필의 인증 기록을 전부 찾아줌 (CourseRepository와 같은 패턴)
Optional<Certification> findByProfileIdAndType(Long profileId, CertificationType type);	"이 프로필의 이 유형(LANGUAGE 등) 인증 기록"을 찾음 — 7-4의 upsert 로직에서 "이미 있는지" 확인할 때 씬

7-3. CertificationResponse · CertificationUpdateRequest

CertificationResponse.java / CertificationUpdateRequest.java

com.passport.certification.dto

졸업 인증 조회·갱신용 데이터 양식.

코드	설명
public record CertificationResponse(Certification.CertificationType type, Certification.CertificationStatus status) { public static CertificationResponse from(Certification certification) { return new CertificationResponse(certification.getType(), certification.getStatus()); } }	ProfileResponse와 같은 "엔티티 → DTO 번역" 패턴. id 없이 type·status 2개만 화면에 돌려줌

<pre>public record CertificationUpdateRequest(List<Item> items) { public record Item(Certification.CertificationType type, Certification.CertificationStatus status) { } }</pre>	새로운 패턴: record 안에 또 다른 record(Item)가 들어있습니다(중첩 record). 어학·봉사·논문 상태를 한 번에 여러 개 보내야 해서, "항목 여러 개를 담은 리스트"를 받는 모양으로 설계했습니다. 실제 요청 예시: {"items": [{"type": "LANGUAGE", "status": "PASS"}, ...]}
---	---

7-4. CertificationService.java — upsert(있으면 수정, 없으면 생성)

CertificationService.java

com.passport.certification.service

list()는 이미 배운 패턴, update()·upsert()가 이 프로젝트에서 가장 새로운 패턴입니다.

코드	설명
<pre>@Service @RequiredArgsConstructor @Transactional(readOnly = true) public class CertificationService {</pre>	ProfileService·CourseService와 같은 3가지 표시(업무처리부품 등록·생성자 자동연결·기본 조회전용모드)
<pre> private final CertificationRepository certificationRepository; private final ProfileRepository profileRepository;</pre>	인증 저장소, 프로필 저장소를 각각 연결
<pre> public List<CertificationResponse> list(Long profileId) { findProfile(profileId); return certificationRepository.findAllByProfileId(profileId).stream() .map(CertificationResponse::from) .toList(); }</pre>	CourseService.list()와 완전히 같은 패턴 — 프로필 존재 확인 후 목록을 스트림으로 DTO 변환
<pre> @Transactional public List<CertificationResponse> update(Long profileId, CertificationUpdateRequest request) { Profile profile = findProfile(profileId); return request.items().stream() .map(item -> upsert(profile, item)) .map(CertificationResponse::from) .toList(); }</pre>	요청으로 들어온 items 목록을 하나씩 순회하며 upsert() 처리한 뒤, 전부 CertificationResponse로 변환해서 반환

<pre>private Certification upsert(Profile profile, CertificationUpdateRequest.Item item) { return certificationRepository.findByProfileIdAndType(profile.getId(), item.type()) .map(certification -> { certification.updateStatus(item.status()); return certification; }) .orElseGet(() -> certificationRepository.save(Certification.builder() .profile(profile) .type(item.type()) .status(item.status()) .build())); }</pre>	upsert = update + insert를 합친 말입니다. 먼저 그 유형(type)의 인증이 이미 있는지 찾아보고 — 있으면(.map) 상태만 바꾸고, 없으면(.orElseGet) 새로 만들어서 저장합니다. "있으면 수정, 없으면 생성"을 한 메서드로 처리하는 흔한 패턴입니다
<pre>private Profile findProfile(Long profileId) { return profileRepository.findById(profileId) .orElseThrow(() -> new BusinessException(ErrorCode.PROFILE_NOT_FOUND)); }</pre>	다른 서비스들과 똑같은 이름·역할의 private 헬퍼 메서드

orElseThrow와 orElseGet의 차이: orElseThrow는 값이 없으면 "에러를 던지고 포기"하지만(4-5, 5-4에서 본 패턴), orElseGet은 값이 없으면 "대신 새로 만들어서 계속 진행"합니다. 같은 Optional을 상황에 따라 다르게 다루는 걸 보여주는 좋은 비교입니다.

7-5. CertificationController.java

CertificationController.java

com.passport.certification.controller

GET·PUT 둘 다 CourseController·ProfileController와 같은 얇은 창구 패턴입니다.

코드	설명
<pre>/** * 인증(JWT) 불기 전 임시 테스트용 컨트롤러 — 소유권 검증 없음. */ @RestController @RequestMapping("/api/v1/profiles/{profileId}/certifications") @RequiredArgsConstructor public class CertificationController { private final CertificationService certificationService; @GetMapping public ApiResponse<List<CertificationResponse>> list(@PathVariable Long profileId) { return ApiResponse.success(certificationService.list(profileId)); } }</pre>	다른 두 컨트롤러와 같은 이유의 메모 + 같은 4가지 표시(창구 표시·기본 주소·생성자 자동연결·클래스 시작)
<pre>private final CertificationService certificationService;</pre>	실제 업무 처리는 서비스에게 위임
<pre>@GetMapping public ApiResponse<List<CertificationResponse>> list(@PathVariable Long profileId) { return ApiResponse.success(certificationService.list(profileId)); }</pre>	CourseController.list()와 같은 패턴 — 목록(List) 반환


```

@PutMapping
public ApiResponse<List<CertificationResponse>>
update(@PathVariable Long profileId,
        @RequestBody
CertificationUpdateRequest request) {
    return
    ApiResponse.success(certificationService.update(profileId,
request));
}

```

ApiResponse<List<CertificationResponse>>처럼 제네릭을 겹쳐 쓸 수도 있습니다 — "성공 여부/메시지로 감싼, CertificationResponse 여러 개가 든 목록"이라는 뜻

여기까지 읽으셨다면 27개 자바 파일 중 단 하나도 빠짐없이 살펴본 것입니다.

8. 자바 코드는 아니지만 알아두면 좋은 파일

파일	역할
build.gradle	이 프로젝트가 어떤 외부 도구(라이브러리)를 쓸지, 자바 버전은 무엇인지 적어놓은 설정 파일. 장보기 목록에 비유 가능
settings.gradle	프로젝트 이름 지정
application.yml	서버 포트(8080), DB 접속 정보 같은 환경 설정
data.sql	서버가 켜질 때마다 자동으로 미리 넣어두는 테스트용 데이터 (지금은 테스트 회원 2명)
.gitignore	git(버전 관리)에 올리지 않을 파일 목록 (빌드 결과물, 비밀 키 파일 등)
postman/PassPort.postman_collection.json	Postman에서 바로 불러와 쓸 수 있는 요청 모음 파일

7. 지금 없는 것 — 다음 단계(인증)에서 생길 파일들

2단계 작업이 시작되면 아래와 같은 파일들이 새로 추가될 예정입니다. 로그인 후에는 Postman 요청마다 로그인 토큰(JWT)을 함께 보내야 합니다.

예상 파일	역할
auth/controller/AuthController.java	회원가입 · 로그인 · 로그아웃 · 내 정보 조회 요청 접수
auth/service/AuthService.java	이메일 중복 체크, 비밀번호 암호화(BCrypt), 로그인 검증 등 실제 처리
auth/dto/*	회원가입/로그인 요청·응답 양식
global/security/JwtTokenProvider.java	로그인 성공 시 발급하는 토큰(JWT)을 만들고 검증하는 도구
global/security/JwtAuthenticationFilter.java	모든 요청마다 토큰이 유효한지 미리 확인하는 문지기
global/config/SecurityConfig.java	어떤 주소는 로그인 없이 허용하고, 어떤 주소는 막을지 정하는 설정

8. 사용한 기술 스택 설명 & 학습 강의 추천

지금까지 "이 코드가 뭘 하는지"를 봤다면, 이번에는 "이걸 어디서 체계적으로 배울 수 있는지"를 정리했습니다. 아래 강의는 전부 인프런(inflearn.com)의 김영한 강사 강의입니다 — 스프링 부트/JPA 분야에서 가장 널리 쓰이는 강의 시리즈입니다.

원래 "김정환" 강사를 언급하셨는데, 실제로 찾아보니 김정환 강사는 React·Vue 같은 프론트엔드 전문 강사였습니다. 이 프로젝트(백엔드: 스프링 부트/JPA)와는 결이 달라서, 아래는 전부 김영한 강사의 백엔드 강의로 정리했습니다. FE(병목님)가 프론트엔드 학습이 필요하시면 김정환 강사 쪽으로 별도 정리해드릴 수 있습니다.

■ ① Java 21 — 언어 기본기

스프링을 배우기 전에 먼저 탄탄해야 하는 자바 자체의 기능들입니다. record(DTO), enum(성적·이수구분), 제네릭(<T>), 스트림(.stream().map()...), Optional(있을 수도 없을 수도 있는 값) 같은 것들이 이 프로젝트 곳곳에 쓰였습니다.

우리 프로젝트에서 쓴 곳 — DTO 전체(record), Grade·ErrorCode(enum), CourseService·CertificationService(스트림), ApiResponse(제네릭)

- 김영한의 실전 자바 - 기본편 (인프런에서 검색)
- 김영한의 실전 자바 - 중급 1편·2편 (인프런에서 검색)

■ ② Spring Boot — 프레임워크 자동 설정

스프링이라는 큰 프레임워크를 "복잡한 설정 없이 바로 쓸 수 있게" 포장해주는 도구입니다. @SpringBootApplication 한 줄로 필요한 설정 대부분이 자동으로 붙습니다.

우리 프로젝트에서 쓴 곳 — PassportApplication.java 전체, build.gradle의 spring-boot-starter-* 의존성들

- 스프링 입문 - 코드로 배우는 스프링 부트, 웹 MVC, DB 접근 기술 (인프런에서 검색)
- 스프링 핵심 원리 - 기본편 — <https://www.inflearn.com/course/스프링-핵심-원리-기본편>
- 스프링 핵심 원리 - 고급편 (인프런에서 검색)

■ ③ Spring Web MVC — REST API 계층

HTTP 요청(GET/POST/PUT/PATCH/DELETE)을 받아서 우리 자바 코드로 연결해주는 계층입니다. @RestController, @GetMapping 같은 어노테이션이 여기 속합니다.

우리 프로젝트에서 쓴 곳 — ProfileController · CourseController · CertificationController

- 스프링 MVC 1편 - 백엔드 웹 개발 핵심 기술 (인프런에서 검색)
- 모든 개발자를 위한 HTTP 웹 기본 지식 — <https://www.inflearn.com/course/http-웹-네트워크>

■ ④ Spring Data JPA + Hibernate — DB 연결

자바 객체(엔티티)와 DB 테이블을 자동으로 연결해주는 기술입니다. SQL을 직접 안 쓰고도 저장·조회·수정·삭제가 가능해집니다. 이 프로젝트에서 가장 핵심적으로 쓰인 기술입니다.

우리 프로젝트에서 쓴 곳 — 모든 *.domain.*(엔티티), 모든 *Repository.java, application.yml의 ddl-auto 설정

- 자바 ORM 표준 JPA 프로그래밍 - 기본편 (인프런에서 검색)
- 실전! 스프링 데이터 JPA (인프런에서 검색)
- 실전! 스프링 부트와 JPA 활용1 - 웹 애플리케이션 개발 (인프런에서 검색)
- 실전! 스프링 부트와 JPA 활용2 - API 개발과 성능 최적화 (인프런에서 검색)

■ ⑤ Lombok — 반복 코드 자동 생성

getter, 생성자 같은 반복 코드를 어노테이션 하나로 자동 생성해주는 도구입니다.

우리 프로젝트에서 쓴 곳 — 모든 엔티티(@Getter, @Builder, @NoArgsConstructor), 모든 서비스(@RequiredArgsConstructor, GlobalExceptionHandler@Slf4j)

- 별도 강의보다는 위 스프링/JPA 강의들 안에서 사용법이 함께 다뤄집니다

■ ⑥ Jackson — JSON 변환

자바 객체 ↔ JSON 문자열을 서로 변환해주는 도구입니다. Grade enum에 @JsonValue/@JsonCreator를 붙여서 내부 이름 A_PLUS 대신 사람이 보기 편한 "A+" 문자열로 주고받도록 커스터마이징했습니다.

우리 프로젝트에서 쓴 곳 — Course.Grade enum

- 보통 스프링 MVC/스프링 부트 강의 안에서 함께 다뤄집니다

■ ⑦ Gradle — 빌드 도구

이 프로젝트를 어떻게 빌드(컴파일+패키징)할지, 어떤 외부 라이브러리를 받아올지 관리하는 도구입니다.

우리 프로젝트에서 쓴 곳 — build.gradle, settings.gradle, gradlew

- 보통 스프링 부트 입문 강의 초반에 설치·사용법이 함께 나옵니다

■ ⑧ H2 데이터베이스 — 개발용 DB

진짜 DB 없이도 바로 테스트할 수 있게 해주는 메모리 DB입니다. 서버를 끄면 데이터가 사라지는 대신, 설치 없이 즉시 쓸 수 있어서 초기 개발 단계에 자주 씁니다.

우리 프로젝트에서 쓴 곳 — application.yml의 datasource 설정, http://localhost:8080/h2-console

- JPA 강의들에서 실습용으로 함께 사용하는 경우가 많습니다

■ ⑨ 계층형 아키텍처 / DTO 분리 / 전역 예외 처리 — 설계 패턴

특정 라이브러리가 아니라 우리가 직접 선택한 코드 "설계 방식"입니다. Controller-Service-Repository로 역할을 나누고, 요청/응답은 DTO로 분리하고, 예러는 한 곳에서 처리하는 방식은 스프링 생태계에서 정석으로 통하는 패턴입니다. 위 JPA/스프링 실전 강의들을 들으면 실습하면서 자연스럽게 체득하게 됩니다.

우리 프로젝트에서 쓴 곳 — 프로젝트 전체 폴더 구조

위 순서대로 들으면 인프런이 공식으로 짜놓은 로드맵과 거의 같습니다 — 스프링 부트와 JPA 실무 완전 정복 로드맵: <https://www.inflearn.com/roadmaps/149> (김영한 강사 프로필: <https://www.inflearn.com/users/@yh>)

Spring Security(로그인 인증)·JWT 관련 강의는 이번 조사에서 김영한 강사 목록에 확인되지 않았습니다. 2단계(인증) 작업을 시작할 때 다시 조사해서 별도로 추천해드리겠습니다 — 확인 안 된 강의를 추측해서 적지는 않았습니다.

여기까지 읽으셨다면 지금 만들어진 27개 자바 파일이 각각 왜 있고 무슨 일을 하는지, 그리고 어떤 기술을 어디서 배우면 되는지까지 파악하신 겁니다.